

Deep Learning – Image Classification Using CNNs and Transfer Learning

Objective :

The objective of this project is to help understand and implement **image classification** using:

- A custom CNN model
- A pre-trained model (Transfer Learning)

A custom CNN model

Environment Setup

- Importing libraries and setting seed.

```
#Importing libraries
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import cifar10
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
#Setting seed for reproducibility
tf.random.set_seed(42)
```

- Loading CIFAR 10 dataset and splitting it into train and test dataset

```
#Loading dataset
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 4s 0us/step

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

```
print("Shape of training dataset : ",X_train.shape)
print("Shape of testing dataset : ",X_test.shape)
```

Shape of training dataset : (50000, 32, 32, 3)
Shape of testing dataset : (10000, 32, 32, 3)

Data Preprocessing

- We are **resizing** images to have 224*224 pixels (with 3 channels RGB) for uniformity in image size.
- **Normalizing** the pixel values converts each pixel value between 0 and 1. This helps us to train our model and converge faster as values are smaller.

```
img_size = 224
resize_and_rescale = keras.Sequential([
    layers.Resizing(img_size, img_size),
    layers.Rescaling(1./255)
])
```

- Applying **Data augmentation** This will enable to train our model on a large amount of data and create a generalized model and prevent overfitting.

```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.2)
])
```

Building CNN model from scratch

Building model

```
model = keras.Sequential([
    #Adding preprocessing layers
    resize_and_rescale,
    data_augmentation,
    #Adding convolution layers along with maxpooling
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    #Flattening the output from maxpooling for forming dense neural network
    layers.Flatten(),
    layers.Dense(128, activation='relu'), #128 here is the output units
    #Output layer with 10 classes
    layers.Dense(10)
])
```

Compiling model

We are using Adam optimizer and since it is a multiclass classification the loss function used is SparseCategoricalCrossentropy

```
#Compiling the model
model.compile(optimizer='adam', loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=['accuracy'])
```

Early Stopping- To prevent overfitting and model stops training once there are no significant improvements seen in the monitored parameter ,in this case 'val_loss'

```
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    min_delta=0.0001,
    patience=5,
    restore_best_weights=True
)
```

Training our model

Here , we are going with 50 epochs with 200 images in each iteration.

```
#Fitting our training dataset
model_history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=200,
    validation_split=0.2,
    callbacks=[early_stopping]
)
```

Output

As we can see our training accuracy is approx. 61% and Validation accuracy is 59%. The loss can also be observed in the below output image.

```
Epoch 38/50
200/200 — 18s 89ms/step - accuracy: 0.5922 - loss: 1.1583 - val_accuracy: 0.5843 - val_loss: 1.2054
Epoch 39/50
200/200 — 18s 90ms/step - accuracy: 0.5969 - loss: 1.1479 - val_accuracy: 0.5865 - val_loss: 1.1900
Epoch 40/50
200/200 — 18s 89ms/step - accuracy: 0.5941 - loss: 1.1475 - val_accuracy: 0.5835 - val_loss: 1.1920
Epoch 41/50
200/200 — 18s 89ms/step - accuracy: 0.5947 - loss: 1.1475 - val_accuracy: 0.5825 - val_loss: 1.1890
Epoch 42/50
200/200 — 18s 89ms/step - accuracy: 0.5970 - loss: 1.1434 - val_accuracy: 0.5789 - val_loss: 1.2039
Epoch 43/50
200/200 — 18s 89ms/step - accuracy: 0.5945 - loss: 1.1438 - val_accuracy: 0.5841 - val_loss: 1.1975
Epoch 44/50
200/200 — 18s 89ms/step - accuracy: 0.6079 - loss: 1.1172 - val_accuracy: 0.5790 - val_loss: 1.2100
Epoch 45/50
200/200 — 18s 89ms/step - accuracy: 0.6016 - loss: 1.1176 - val_accuracy: 0.5893 - val_loss: 1.1827
Epoch 46/50
200/200 — 18s 89ms/step - accuracy: 0.6051 - loss: 1.1166 - val_accuracy: 0.5835 - val_loss: 1.2022
Epoch 47/50
200/200 — 18s 89ms/step - accuracy: 0.6056 - loss: 1.1091 - val_accuracy: 0.5854 - val_loss: 1.1955
Epoch 48/50
200/200 — 18s 90ms/step - accuracy: 0.6119 - loss: 1.1071 - val_accuracy: 0.5938 - val_loss: 1.1796
Epoch 49/50
200/200 — 18s 90ms/step - accuracy: 0.6054 - loss: 1.1179 - val_accuracy: 0.5889 - val_loss: 1.1775
Epoch 50/50
200/200 — 18s 89ms/step - accuracy: 0.6103 - loss: 1.1086 - val_accuracy: 0.5923 - val_loss: 1.1683
```

Model Evaluation

```
cnn_test_loss, cnn_test_acc = model.evaluate(X_test, y_test, verbose=2)
print("Test Loss : {:.2f}" .format(cnn_test_loss))
print("Test Accuracy : {:.2f}" .format(cnn_test_acc))
```

```
313/313 - 2s - 6ms/step - accuracy: 0.5934 - loss: 1.1520
Test Loss : 1.15
Test Accuracy : 0.59
```

Transfer Learning – Using InceptionV3 as base model

Importing Libraries And Configuring base model

```
#Importing libraries for our pre-trained model
from tensorflow.keras.applications.inception_v3 import InceptionV3
from tensorflow.keras.applications.inception_v3 import preprocess_input
```

Configuring our base model as InceptionV3, here we are setting **include_top=False** to remove the fully connected layers as we will be adding it ourselves during model building.

```
#Configuring our base model as InceptionV3 and removing fully connected layers by setting include_top=False
base_model = InceptionV3(include_top=False, weights='imagenet', input_shape=(224, 224, 3))
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception_v3/inception_v3_weights_tf_dim_ordering_tf_kernels_notop.h5
87910968/87910968 ————— 0s 0us/step

Freezing the layers in base model allows us to use the pre-trained weights of the model.

```
# Freeze all layers
for layer in base_model.layers:
    layer.trainable = False
```

Building model using InceptionV3

Building model

```
# Building model with base model
cnn_inceptionv3 = keras.Sequential([
    resize_and_rescale,
    data_augmentation,
    base_model,
    #You can use Flatten layer over here but to reduce the spatial dimensions and prevent overfitting GlobalAveragePooling is better option
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Compiling Model

```
# Compiling model
cnn_inceptionv3.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Training our model

```
# Training the model
cnn_inceptionv3_model_history = cnn_inceptionv3.fit(
    X_train, y_train,
    epochs=50,
    batch_size=200,
    validation_split=0.2,
    callbacks=[early_stopping]
)
```

Output

As we can observe the model took less epochs to reach a state of high accuracy approx. 70% for training and 80% for validation dataset.

```
Epoch 10/50
200/200 — 61s 306ms/step - accuracy: 0.6767 - loss: 0.9108 - val_accuracy: 0.7919 - val_loss: 0.5996
Epoch 11/50
200/200 — 61s 306ms/step - accuracy: 0.6775 - loss: 0.9086 - val_accuracy: 0.7963 - val_loss: 0.5904
Epoch 12/50
200/200 — 61s 305ms/step - accuracy: 0.6886 - loss: 0.8924 - val_accuracy: 0.7945 - val_loss: 0.5946
Epoch 13/50
200/200 — 61s 305ms/step - accuracy: 0.6877 - loss: 0.8895 - val_accuracy: 0.7844 - val_loss: 0.6108
Epoch 14/50
200/200 — 61s 305ms/step - accuracy: 0.6843 - loss: 0.8967 - val_accuracy: 0.7947 - val_loss: 0.5903
Epoch 15/50
200/200 — 61s 305ms/step - accuracy: 0.6894 - loss: 0.8771 - val_accuracy: 0.8019 - val_loss: 0.5752
Epoch 16/50
200/200 — 61s 305ms/step - accuracy: 0.6870 - loss: 0.8832 - val_accuracy: 0.7910 - val_loss: 0.5965
Epoch 17/50
200/200 — 61s 305ms/step - accuracy: 0.6897 - loss: 0.8775 - val_accuracy: 0.7988 - val_loss: 0.5846
Epoch 18/50
200/200 — 61s 304ms/step - accuracy: 0.6920 - loss: 0.8812 - val_accuracy: 0.7997 - val_loss: 0.5763
Epoch 19/50
200/200 — 61s 304ms/step - accuracy: 0.7024 - loss: 0.8580 - val_accuracy: 0.7994 - val_loss: 0.5793
Epoch 20/50
200/200 — 61s 304ms/step - accuracy: 0.6978 - loss: 0.8565 - val_accuracy: 0.7959 - val_loss: 0.5795
```

Model Evaluation

```
#Model Evaluation
cnn_inceptionv3_test_loss,cnn_inceptionv3_test_acc= cnn_inceptionv3.evaluate(X_test, y_test, verbose=2)
print("Test Loss : {:.2f}" .format(cnn_inceptionv3_test_loss))
print("Test Accuracy : {:.2f}" .format(cnn_inceptionv3_test_acc))
```

313/313 - 19s - 62ms/step - accuracy: 0.7999 - loss: 0.5822

Test Loss : 0.58

Test Accuracy : 0.80

Custom CNN vs Transfer Learning

Visual Representation

Let's plot some visuals to see the difference between the accuracy and the loss

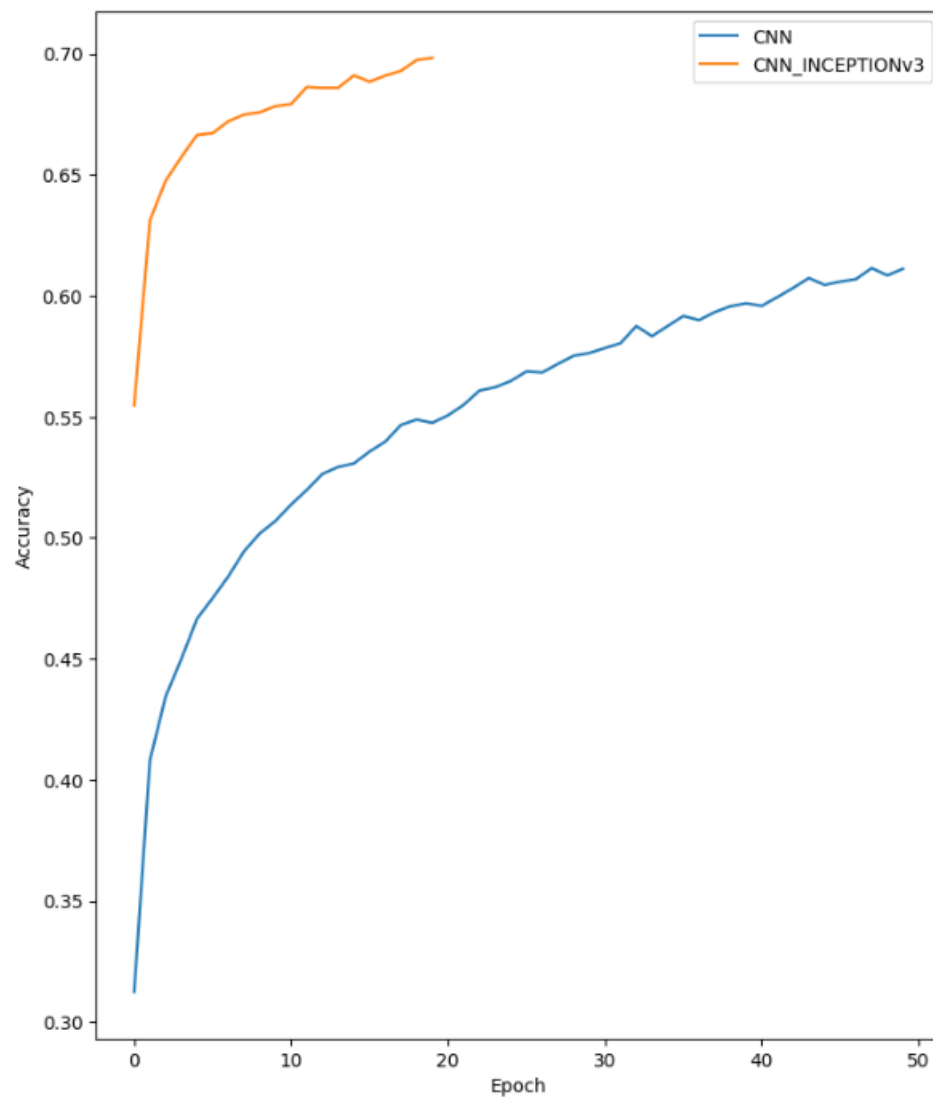
```
plt.figure(figsize=(18,10))
plt.subplot(1,2,1)
plt.plot(model_history.history['accuracy'],label = 'CNN')
plt.plot(cnn_inceptionv3_model_history.history['accuracy'],label = 'CNN_INCEPTIONv3')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1,2,2)
plt.plot(model_history.history['loss'],label = 'CNN')
plt.plot(cnn_inceptionv3_model_history.history['loss'],label = 'CNN_INCEPTIONv3')
plt.xlabel('Epoch')
plt.ylabel('Loss')

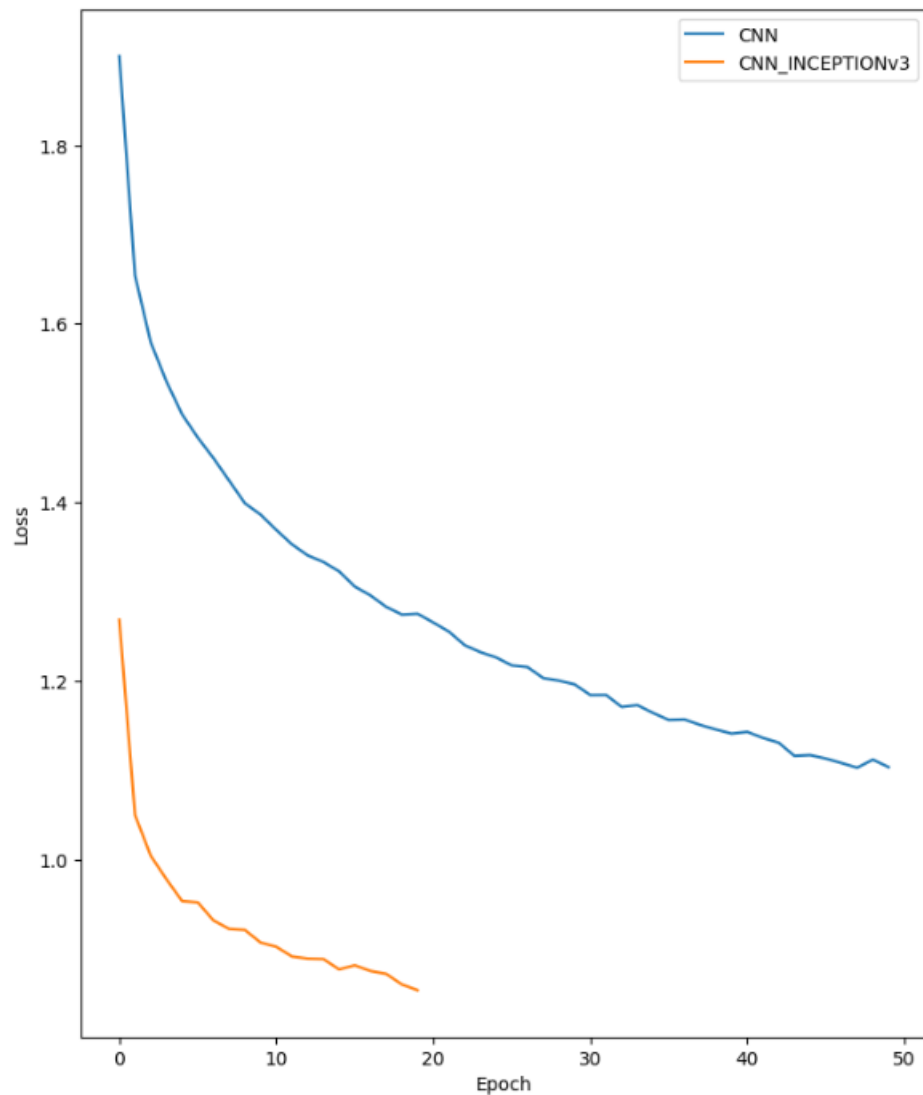
plt.legend()
plt.show()
```

Output

Plotting accuracy- Requiring lesser epochs our cnn model along with inceptionv3 as base model performs quite well as compared to the custom one.



Plotting loss : Similar results can be seen over here, our transfer learning is performing better.



Model Evaluation Results

As we can see, in very few epochs cnn with pre-trained model gives us better results as compared to building our very own cnn model.

Early stopping helps us prevent overfitting , therefore we can see results for epochs until any further improvement is seen in validation loss

To observe in below case, the accuracy increased by 30% and the loss has decreased by 50% which seems to be a significant improvement.

```
#Comparing Accuracy and Loss for CNN vs CNN with transfer learning
results_dict = {
    'Accuracy': [cnn_test_acc, cnn_inceptionv3_test_acc],
    'Loss': [cnn_test_loss, cnn_inceptionv3_test_loss]
}

results = pd.DataFrame(results_dict, index=['CNN', 'CNN+INCEPTIONv3'])

print("Model Evaluation results: \n", results)
```

Model Evaluation results:

	Accuracy	Loss
CNN	0.5934	1.151953
CNN+INCEPTIONv3	0.7999	0.582160

Displaying sample Images

Here we are plotting images with their true and predicted labels.

```
#Model prediction
y_pred = cnn_inceptionv3.predict(X_test)
```

313/313  19s 54ms/step

+ Code

+ Markdown

```
#Displaying sample predictions with images
```

```
def plot_images(index):
    img = X_test[index]
    true_label = class_names[int(y_test[index])]#Actual value
    pred_label = class_names[np.argmax(y_pred[index])]#Predicted value

    plt.imshow(img)
    plt.title(f"True: {true_label} vs Pred: {pred_label}")
    plt.axis('off')
    plt.show()

#Plotting first 5 images
for i in range(5):
    plot_images(i)
```

Output

True: cat vs Pred: dog



True: ship vs Pred: ship



True: ship vs Pred: ship



True: airplane vs Pred: airplane



True: frog vs Pred: frog



Conclusion

1. Model Performance

- The CNN model we created has low accuracy as compared to CNN model with transfer learning (in this case InceptionV3) and takes time to converge with higher number of epochs

2. Transfer Learning

- Transfer Learning is effective approach as the pre-trained model has higher ability and trained weights to improve our model performance.
- Early layers capture low-level features, while deeper layers recognise high-level patterns.
- We need to consider overfitting factor while using pre-trained models as it can sometimes fail to generalize the model.(If your training accuracy is high but the validation/test accuracy is low-your model is overfitting)

3. Key learnings

- Use randomseed to reproduce same results over time.
- Early stopping help us to stop the model training once any significant changes are seen in the monitored parameters.
- Using pre-trained model gives better accuracy and faster results.